26-1 지능형로보틱스 최종 프로젝트

자율주행 로봇 기반 비상구 및 대피로 차단 탐지 시스템

팀 올라프
2371023 김현영

목차

01

서론

문제 배경
프로젝트 목표

02

환경 구축

사용 기술 스택

03

시스템 설계

전체 아키텍처
ROS2 Node 구성
토픽 흐름도

04

구현 및 결과

지도 생성 및 시뮬레이션 환경 구성
자율 순찰 노드 구현
지속 감지 기반 장애물 판단
시각화

05

데모

서론: 문제 배경 및 필요성



‘제천 참사’ 겪고도…비상구 관리 여전히 엉망

입력 2018.03.01 (06:44) | 수정 2018.03.02 (10:02) 출처: KBS 뉴스 성용희 기자

'화재로 10명 부상' 소공동 호텔, 과거 '비상구 적치물' 신고돼

출처: YTN 신귀혜 기자 2026.03.18. 오후 5:54.

- 🔴 실내 건물 비상구·복도에 박스, 의자, 케이블 등 적치물 방치 일상화
- 🔴 화재·재난 발생 시 대피 동선 차단 → 인명 피해로 직결
- 🔴 관리자 수동 순찰에 의존 → 야간·공휴일 점검 공백 발생

프로젝트 목표: "이동 로봇으로 점검을 자동화하자!



자율 순찰

TurtleBot3가 사전에 정의된
점검 구역을 따라 이동하며
비상구와 중앙 구역을 순찰



지속 감지 기반 장애물 판단

LiDAR 센서 데이터를 바탕으로
일시 감지가 아닌 지속 감지를 통해
비상구 차단 여부를 판단



시각화 및 상태 모니터링

점검 결과를 RViz Marker와
터미널 로그로 확인하여
현재 상태를 직관적으로 모니터링

사용 기술 스택

Category	Technology / Tools	Description
OS / Framework	Ubuntu 22.04 LTS, ROS2 Humble	ROS2 기반 로봇 시스템 개발 환경
Simulator	Gazebo Classic	복도 및 비상구 차단 상황 시뮬레이션
Robot	TurtleBot3 Waffle Pi	순찰 및 센서 데이터 수집용 이동 로봇
Mapping / Localization	SLAM Toolbox, AMCL	지도 생성 및 지도 기반 위치 추정
Navigation	Custom Patrol Control	/odom 기반 위치 추정 및 /cmd_vel 이동 제어
Sensor	LiDAR /scan	장애물 거리 측정 및 차단 여부 판단
Custom Node	scan_blockage_judge	LiDAR 기반 장애물 판단 및 지속 감지 상태 확정
Visualization	RViz2	지도, 로봇, 라이다, TF 시각화
Language	Python rclpy	ROS2 노드 및 판단 로직 구현

시스템 설계



[ROS2 Node 구성]

1. corridor_patrol (로봇 순찰 및 이동 제어)

INPUT → /odom, /scan, /blockage_state

OUTPUT → /cmd_vel, /current_zone, /inspection_target, /inspection_active

2. scan_blockage_judge (LiDAR 기반 장애물 판단)

INPUT → /scan, /inspection_active, /inspection_target

OUTPUT → /blockage_state

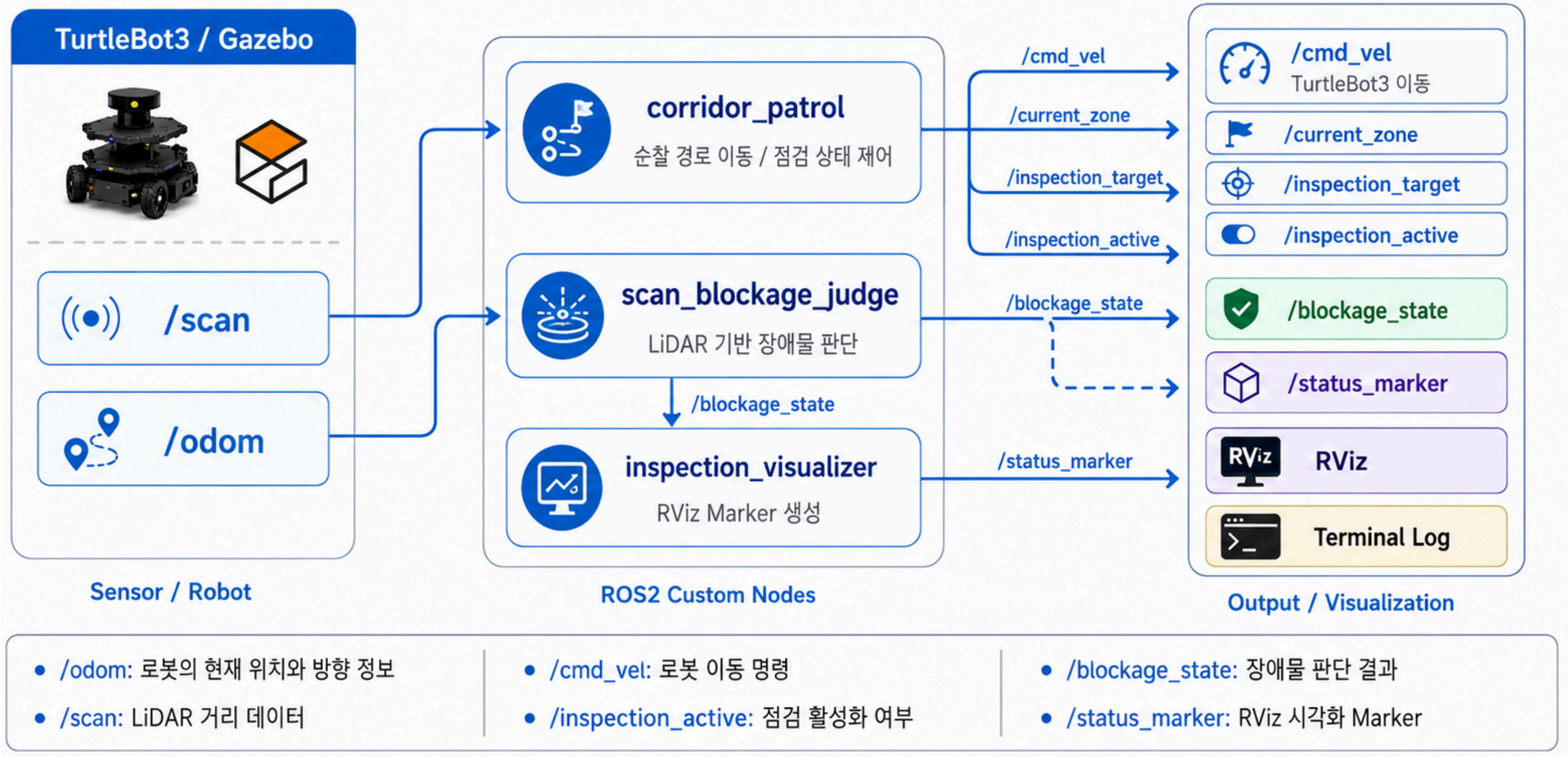
3. inspection_visualizer (판단 결과 시각화)

INPUT → /blockage_state, /current_zone

OUTPUT → /status_marker

Topic 흐름도

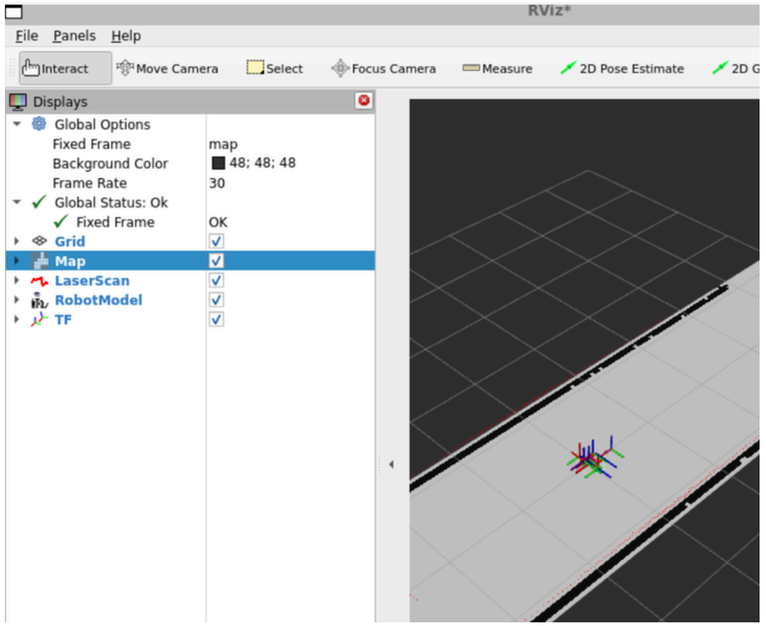
센서 데이터 수집부터 장애물 판단 및 시각화까지의 ROS2 Topic 기반 데이터 흐름

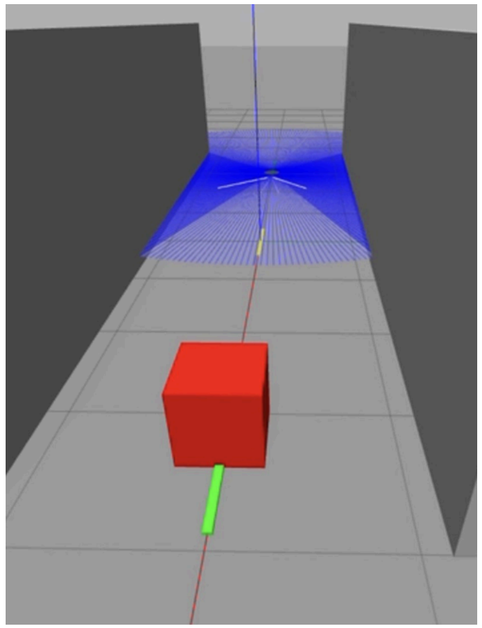


구현 및 결과

🟥장애물 🟨중앙 점검 구역 🟩비상구 점검 구역 ⬛복도

1) 지도 생성 및 시뮬레이션 환경 구성





Gazebo에서 비상구 점검 구역과 장애물이 포함된 복도 환경을 구성하고,
RViz를 통해 로봇의 위치, LiDAR 데이터, 지도 정보를 함께 확인함.

2) 자율 순찰 노드 구현

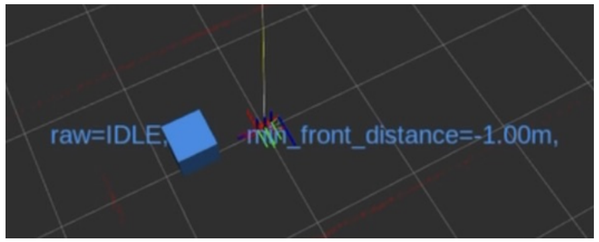
비상구 1
↓
중앙
↓
비상구 2

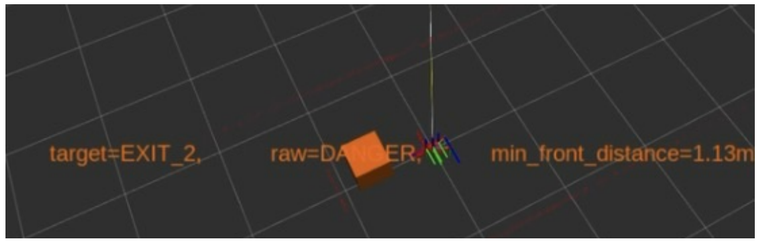
데모에서는 **비상구 2**에
장애물이 위치한 상황으로 가정하였음.

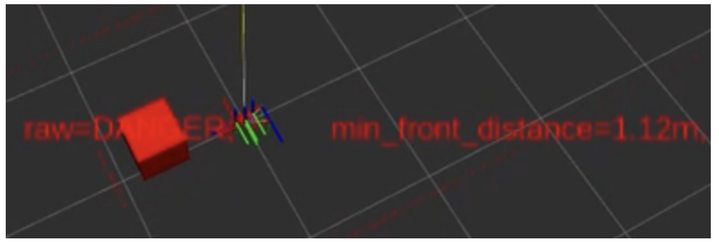
구현 및 결과

3) 지속 감지 기반 장애물 판단

- PATROLLING(판단 대기) → DANGER_CANDIDATE(위험 후보 감지) → CONFIRMED_DANGER(위험 확정)
- 전방 ±60도 LiDAR 범위에서 장애물이 1.4m 미만으로 감지되면 DANGER_CANDIDATE로 전환되며,
- 해당 상태가 2초 이상(데모) 지속될 경우 CONFIRMED_DANGER로 확정됨.







4) 시각화 (+ 개선 방향)

- 로봇의 순찰 진행 상황과 장애물 판단 결과를 RViz, 터미널 로그를 통해 동시에 확인할 수 있도록 구성하였음.
- 시간 관계상 본 프로젝트에서는 시도하지 못했지만, CONFIRMED_DANGER 발생 시 관리자에게 알림을 전송하고 웹 대시보드에서 로봇 위치, 점검 구역, 장애물 판단 상태를 실시간으로 확인할 수 있도록 발전시킬 수 있음.

어려웠던 점

- 개인 노트북 환경에서는 실행이 무겁고 끊김이 발생하였다. 여러 노트북을 대여해 시도해보았지만, 매끄러운 실행에 어려움이 있어 최종적으로 ECC B204 컴퓨터실 환경에서 작업하였다. 컴퓨터실 환경은 하루가 지나면 세팅이 초기화 되어, 작업할 때마다 ROS2, Gazebo, TurtleBot3 관련 환경 설정을 반복해야 하는 문제가 있었다,,

- 이동 중 LiDAR가 벽이나 주변 구조물을 감지하면서 장애물로 오인될 가능성이 있어, 점검 구역에서만 판단이 활성화되도록 로직을 조정하였다.

- 순간적인 센서 값 변화로 인한 오탐을 줄이기 위해 거리 기준과 지속 감지 시간을 여러 번 조정하였다.

데모 영상